

Projet de Simulation - M1 Actuariat - 2024-2025

Groupe G01

Résumé

On considère une assurance qui assure N motards. On note R le remboursement total à effectuer sur toute cette période et pour tous les motards et on cherche à approximer, par une méthode de Monte Carlo, l'espérance conditionnelle

$$m(s) = \mathbb{E}(R - s | R > s),$$

où s est un seuil donné.

Les simulations devront être effectuées avec le logiciel R, en faisant, dans un deuxième temps, appel à une partie de code sous C/C++.

Ce document présente le code et les méthodes implémentées par le groupe G01 pour répondre à cette problématique.

Auteurs

- Salimata DIOUF
- Christ Ange Dylan KOUAME
- Aboubakar Mohamed OUATTARA
- Souleymane OUATTARA

Professeur

- Alexis BIENVENÜE



Table des matières

1	Organisation du travail en groupe	5
1.1	Organisation	5
1.2	Outils utilisés	5
2	Choix des paramètres	5
2.1	<code>rremb</code>	5
2.2	<code>simulate_weather_vectorize</code>	7
2.3	probabilités d'accident	7
3	Rajout de la fonction intermédiaire	8
4	Implémentation de <code>rremb</code>	9
4.1	Simulation de <code>rremb</code> par la méthode de rejet	9
4.2	Simulation de <code>rremb()</code> par la méthode d'inversion	11
4.2.1	Détermination de la fonction de répartition	12
4.2.1.1	Calcul de la constante	12
4.2.1.2	Calcul de F	12
4.2.2	Détermination de la fonction de répartition inverse	13
4.2.3	Création de la fonction <code>rremb()</code>	13
4.2.3.1	Simplification de F^{-1}	14
4.2.3.2	Implémentation de <code>rremb()</code>	15
5	Implémentation de <code>msa</code> et <code>msb</code>	16
5.1	explications préalables	16
5.1.1	la probabilité d'accidents	16
5.1.2	l'espérance conditionnelle par monte carlo	17
5.2	La fonction <code>msa</code> : explication de l'algorithme	17
5.2.1	Initialisation des variables :	17
5.2.2	Boucle de simulation Monte Carlo	18
5.2.3	Calcul de l'espérance conditionnelle	18
5.2.4	valeur de sortie de la fonction <code>msa</code>	19
5.3	La fonction <code>msb</code> : explication de l'algorithme	20
5.3.1	Initialisation des variables	20
5.3.2	Problématique par rapport à la méthode initiale	21
5.3.3	Méthode alternative	21
5.3.4	Calcul de l'espérance conditionnelle	22
5.3.5	Valeur de sortie de la fonction <code>msb</code>	23
6	Interfaçage de <code>msb</code> en C++	23
6.1	Algorithmie	23
6.2	Résultats	25
7	Analyse du modèle	26
7.1	Influence des paramètres	26
7.2	Cas d'application du modèle	28
7.3	Pistes d'amélioration	30

Table des figures

1	Histogramme $rremb$ $\alpha < x_0$	6
2	Histogramme $rremb$ $\alpha > x_0$	6
3	Histogramme $rremb$ $\alpha = x_0$	7
4	script de <code>simulate_weather_vectorised</code>	9
5	histogramme de la répartition de la météo sur 100 ans	9
6	Courbe représentative f^*	10
7	algorithme de rejet uniforme	10
8	Histogramme $rremb()$	11
9	Temps d'exécution réjet	11
10	code de $rremb$	15
11	Histogramme de $rremb$	16
12	Temps d'exécution Méthode par inversion	16
13	code de msa	19
14	Paramètres d'exécution de l'exemple	19
15	Résultats de la fonction et Temps d'exécution de msa	20
16	script initial de la fonction <code>msb</code>	21
17	Partie n°1 <code>msb.c</code>	23
18	Partie n°2 <code>msb.c</code>	23
19	Partie n°3 <code>msb.c</code>	24
20	Partie n°4 <code>msb.c</code>	24
21	Partie n°5 <code>msb.c</code>	25
22	Temps d'exécution n°1 <code>msb</code>	25
23	Temps d'exécution n°1 <code>msb.c</code>	25
24	Temps d'exécution n°2 <code>msb</code>	25
25	Temps d'exécution n°2 <code>msb.c</code>	25
26	Valeurs $m(s)$ avec <code>msb</code>	26
27	Valeurs $m(s)$ avec <code>msb.c</code>	26
28	variation de msa et msb en fonction de la probabilité de d'accident en cas de pluie	27
29	Variation de msa et msb en fonction du paramètre x_0	27
30	variation de msa et msb en fonction du paramètre η	27
31	variation de msa et msb en fonction du seuil s	28
32	variation de msa et msb en fonction de p_{NP}	28
33	Histogramme comparatif des différents remboursements pour chaque hypothèse.	29
34	Table comparative des différents remboursements pour chaque hypothèse	30
35	Fonctionnement Excédent de sinistre	30

1 Organisation du travail en groupe

1.1 Organisation

Le travail a été reparti comme suit :

Dylan était chargé de la rédaction des différents code afin de pouvoir simuler la fonction *rremb*, Salimata quant-à elle devait se charger de la rédaction du code de *msa* ainsi que des fonctions intermédiaires que sont celles pour parvenir à simuler la météo et calculer le vecteur des accidents. Aboubakar lui devait rediger le rapport et s'occuper de l'implémentation de la fonction *msb*. Et pour finir, Souleymane devait se charger de l'interfaçage en C++ en R de *msb*.

Bien que ces tâches étaient déjà bien définies celà ne nous n'empêchait pas de nous entraider en cas de blocages ou de difficultés.

Nous nous retrouvions quasiment chaque Week-end pour travailler sur le projet.

1.2 Outils utilisés

Nous avons utilisé Google Collab pour la rédaction collaborative du code sous R. En ce qui concerne la rédaction du rapport, nous avons utilisé LaTeX via Overleaf pour la mise en forme et de la documentation en ligne.

2 Choix des paramètres

Le choix des paramètres a principalement été motivé par des réflexion logique et des recherches

2.1 rremb

L'objectif étant juste de nous assurer que le code de simulation de la loi de remboursements fonctionnait correctement, nous avons fixés des paramètres α, η et x_0 qui respectaient les conditions d'évaluations du code :

$$0 < \alpha < 3, \quad 0 < x_0 < 3, \quad \eta > 3$$

A η constant égal à 3.5 , nous avons analysés plusieurs cas de figures :

- le cas où $\alpha < x_0$: Dans ce cas nous avons utilisés $\alpha = 1.5$ et $x_0=2$ et nous avons obtenus la representation suivante :

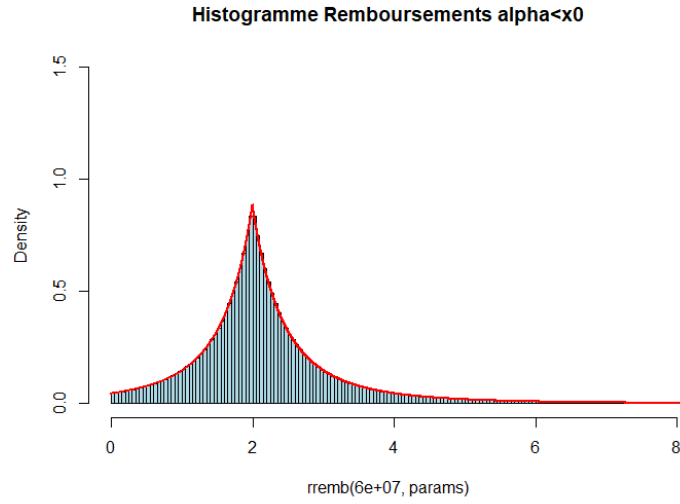


FIGURE 1 – Histogramme $rremb \alpha < x_0$

- le cas où $\alpha > x_0$: Dans ce cas de figure nous avons choisis $\alpha = 2$ et $x_0=1.5$ et nous avons obtenus la representation suivante :

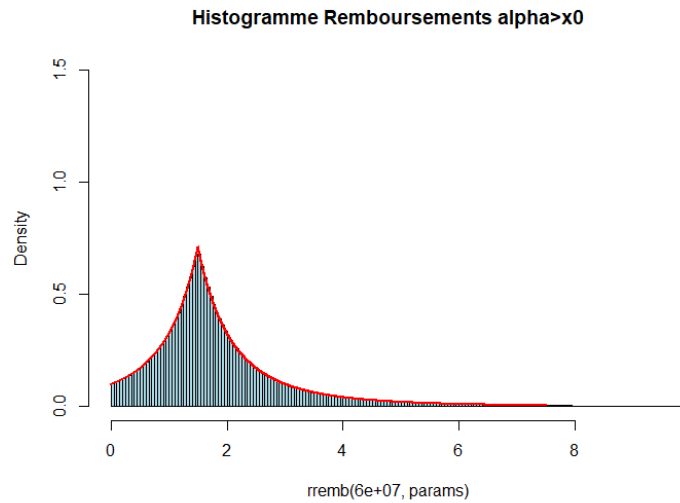


FIGURE 2 – Histogramme $rremb \alpha > x_0$

- Enfin le cas d'égalité : Nous avons utilisés $\alpha = 1$ et $x_0=1$ et nous avons obtenus cette representation similaire aux deux autres mais plus centrée en $x_0=1$.

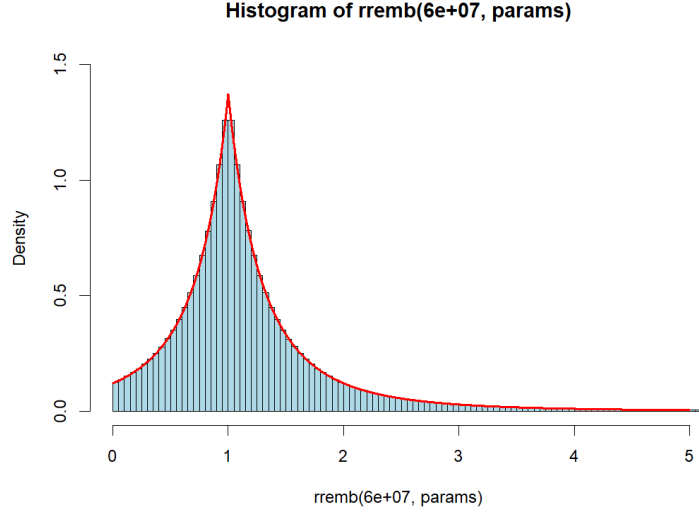


FIGURE 3 – Histogramme $rremb \alpha = x_0$

Ainsi le choix des paramètres ne revêt pas vraiment d'importance pour la simulation de la loi de Remboursements.

Dans les parties suivantes abordant la construction de la fonction $rremb()$ par différentes méthodes, nous nous placerons dans le dernier cas de figures pour les sorties graphiques ($\alpha = 1$, $x_0=1$ et $\eta= 3.5$)

2.2 *simulate_weather_vectorize*

cette fonction a été utilisée principalement afin de pouvoir simuler la météo annuelle que ce soit sous la première hypothèse ou la seconde, bien sûr en changeant l'algorithmie.

2.3 probabilités d'accident

Afin d'assurer un certains réalisme et effectuer des tests de nos algorithmes, nous choisirons des valeurs spéciales pour les paramètres relatif aux transitions météorologiques et probabilités d'accidents :

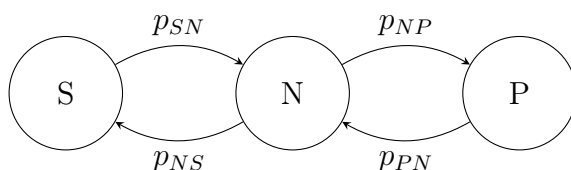
- Nous partons de la logique assez acceptable qu'il y'a autant de chances de rester dans l'état **ensoillé** que de passer à un état **Nuageux** et idem pour l'état **Pluvieux**.
De même il est acceptable qu'il y'ait autant de chances d'aller à "P" qu'à "S", donc nous avons choisi comme paramètres pour le calcul de **msa** et **msb** : $p_{SN} = 1/2$, $p_{NS} = 1/4$, $p_{NP} = 1/4$ et $p_{PN} = 1/2$.
- En ce qui concerne les probabilités d'accidents en fonction des météo, nos recherches nous ont permis d'obtenir que des selon des estimations 5 à 10% des motards sur route peuvent être impliqués dans un accident sur une journée donnée, là où on a 2% en temps "Nuageux" et 1% en période "Ensoleillée".
Ainsi dans la suite nous considérerons : $p_{acc}=c(p_S = 0.01, p_N = 0.02$, $p_P = 0.07)$

3 Rajout de la fonction intermédiaire

la fonction est réalisée à l'aide de la chaîne de Markov fournie dans la consigne du projet.

On se trouve dans un modèle avec 3 états que sont enSoleillé représenté par le chiffre 1, Nuageux par le 2 et Pluvieux par le 3.

$$\mathbf{E} = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}$$



On crée une matrice de transition conforme à la chaîne,

$$\mathbf{P} = \begin{pmatrix} 1 - p_{SN} & p_{SN} & 0 \\ p_{NS} & 1 - (p_{NS} + p_{NP}) & p_{NP} \\ 0 & p_{PN} & 1 - p_{PN} \end{pmatrix}$$

de vecteur initial :

$$\pi_0 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}$$

qui représente bien le soleil défini comme état initial de la chaîne. On commence dans un premier temps par récupérer les probabilités de transition qui sont fournies par l'utilisateur, dans des variables de sorte à rendre le code plus compréhensible, et aussi pour pouvoir créer la matrice de transition.

Ensuite on crée un vecteur météo de taille **days** (days correspondant au nombre de jours) qui prend 1 (Soleil) comme première valeur, et les autres valeurs sont choisies de façon aléatoire par rapport au jour précédent grâce à la fonction `sample` qui va faire un choix de façon aléatoire suivant les probabilités à l'intérieur de **current_state**, qui lui représente la ligne dans la matrice de transition relative à l'état précédent.

```

simulate_weather_vectorized <- function(days, params) {
  #recuperation des probabilites de transition
  pSN <- params$ptrans[1]
  pNS <- params$ptrans[2]
  pNP <- params$ptrans[3]
  pPN <- params$ptrans[4]
  #creation de la matrice de transition
  ptrans <- matrix(c(
    1 - pSN, pSN, 0,
    pNS, 1 - (pNS + pNP), pNP,
    0, pPN, 1 - pPN
  ), nrow = 3, byrow = TRUE)
  #creation d'un vecteur de taille days qui a pour premiere valeur 1 representant le soleil
  weather <- integer(days)
  weather[1] <- 1
  #on crée une boucle sur le nombre de jour et qui affecte un nombre entre 1 et 3 au jour suivant, en
  #fonction des probabilites de transition de la chaîne et de l'etat precedent
  for (day in 2:days) {
    current_state <- weather[day - 1]
    weather[day] <- sample(1:3, 1, prob = ptrans[current_state, ])
  }
  return(weather)
}

```

FIGURE 4 – script de `simulate_weather_vectorised`

et l'on observe une certaine cohérence au niveau des proportions obtenues pour des probabilités de transitions toutes fixées à 1. On observe donc sur 100 ans soit environ 36500 jours, une proportion d'environ 9000 jours ensoleillés, 18000 nuageux et 9000 pluvieux pour les probas :

$$p_{SN} = 1/2 ; p_{NS} = 1/4 ; p_{NP} = 1/4 ; p_{PN} = 1/2$$

ce qui est plutôt réaliste.

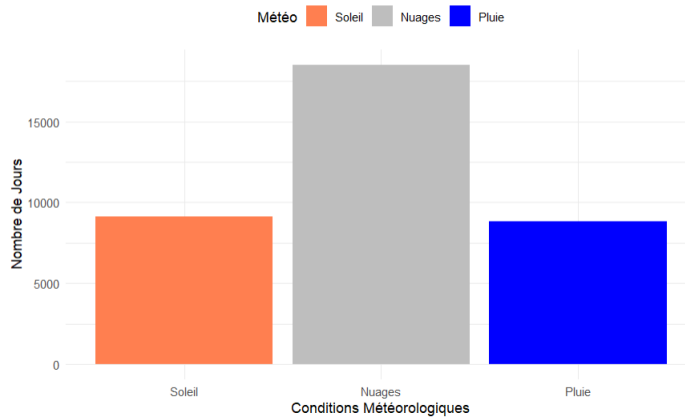


FIGURE 5 – histogramme de la répartition de la météo sur 100 ans

4 Implémentation de *rremb*

4.1 Simulation de *rremb* par la méthode de rejet

Nous avons effectué dans un premier temps une simulation par rejet, spécialement par rapport à la loi uniforme. Pour cela, nous avons procédé comme suit :

- Etude de la fonction à la main : Cette étude nous a permis de voir que la fonction était symétrique par rapport à x_0 , y atteignait son maximum et tendait vers 0 en $+\infty$
- Allure sous **R** de f^* (avec les paramètres mentionnées plus haut) pour confirmer nos idées et voir à partir de quel point peut on considérer la fonction comme nulle en vue du rejet.

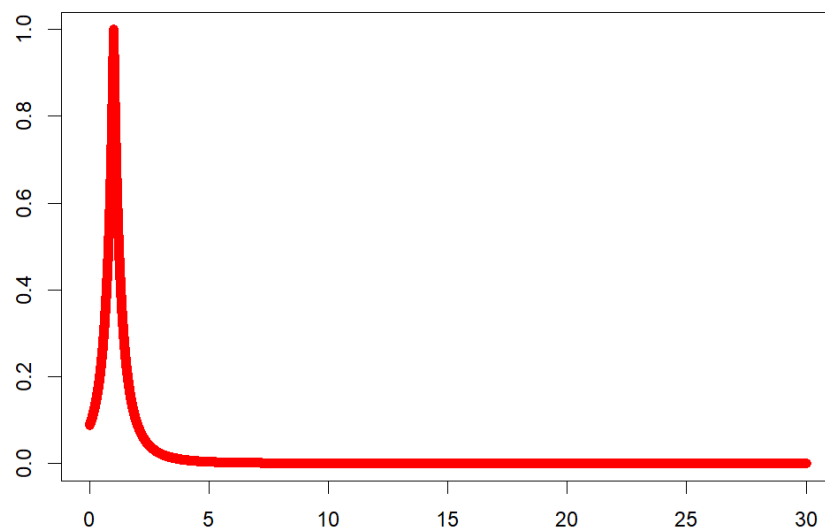


FIGURE 6 – Courbe représentative f^*

- Choix de l'intervalle $[0, 5x_0]$ comme intervalle raisonnable en dehors duquel considérer la fonction comme nulle, vu la vitesse à laquelle elle tend vers 0.
- Code de la fonction `remb` :

```
f_etoile<-function(x,alpha,eta,x0){
  return(exp(-eta*log(alpha+abs(x-x0)))*1*(x>=0))
}
f_etoile<-Vectorize(f_etoile)

rremb<-function(n,params){
  alpha<- params$alpha
  eta<- params$eta
  x0<- params$x0
  vec<-numeric(n)
  for(i in 1:n){
    x<-runif(1,0,5*x0)
    y<-runif(1,0,1/alpha^eta)
    while(y>f_etoile(x,alpha,eta,x0)){
      x<-runif(1,0,5*x0)
      y<-runif(1,0,1/alpha^eta)
    }
    vec[i]<-x
  }
  return(vec)
}
```

FIGURE 7 – algorithme de rejet uniforme

- Dans le code precedent on a généré

$$x \sim \mathcal{U}([0, 5x_0]) \quad \text{et} \quad y \sim \mathcal{U}([0, 1/\alpha^\eta])$$

car f^* atteint son maximum en $f^*(x_0)$ qui vaut $1/\alpha^\eta$ et pour le reste nous avons suivi la logique apprise en cours.

- Visualisation du résultat :

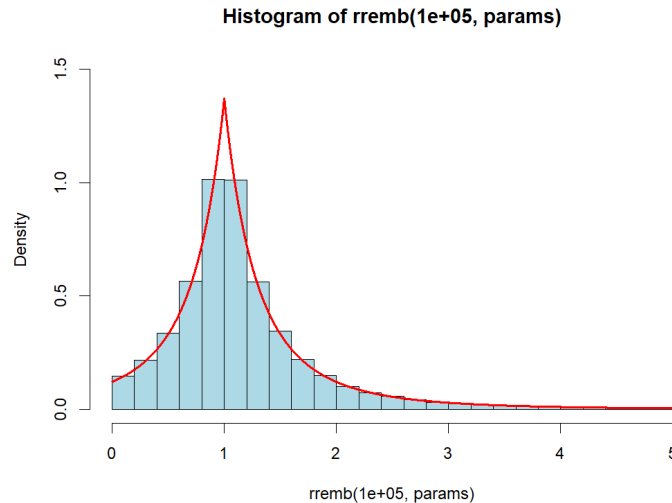


FIGURE 8 – Histogramme $rremb()$

- Le graphe précédent représente la distribution des valeurs retournées par $rremb()$ sur lequel on superpose la fonction f^* normalisée dont l'expression est la suivante :

$$f(x) = \frac{\eta - 1}{\frac{2}{\alpha^{\eta-1}} - \frac{1}{(\alpha+x_0)^{\eta-1}}} \cdot f^*(x)$$

(les calculs de la constante de normalisation seront détaillés plus bas)

- La méthode de rejet permet de simuler correctement la loi de remboursement mais s'avère assez lente.

```
> set.seed(123)
> system.time(rremb(100000,params))
utilisateur      système      écoulé
      14.59         0.34       15.17
> |
```

FIGURE 9 – Temps d'exécution réjet

Etant globalement insatisfait, nous avons décidé d'explorer une autre piste qui s'est avérée plus fructueuse : La méthode par inversion.

4.2 Simulation de $rremb()$ par la méthode d'inversion

Voici les étapes principales :

- Détermination de la fonction de répartition F .
- Inversion pour obtenir F^{-1} .
- Création de la fonction *rremb* pour générer des échantillons.

4.2.1 Détermination de la fonction de répartition

Déterminons d'abord au préalable le facteur de normalisation de $f^*(x)$, indispensable à l'obtention la fonction de répartition.

4.2.1.1 Calcul de la constante

Soit $\alpha > 0$, $x_0 > 0$ et $\eta > 3$.

On a :

$$\begin{aligned}
 \int f^*(x) dx &= \int_0^{+\infty} \frac{1}{(\alpha + |x - x_0|)^\eta} dx \\
 &= \int_0^{x_0} \frac{1}{(\alpha + x_0 - x)^\eta} dx + \int_{x_0}^{+\infty} \frac{1}{(\alpha + x - x_0)^\eta} dx \\
 &= \int_{\alpha+x_0}^{\alpha} -\frac{1}{x^\eta} dx + \int_{\alpha}^{+\infty} \frac{1}{x^\eta} dx \\
 &= \frac{1}{\eta-1} \left(\frac{2}{\alpha^{\eta-1}} - \frac{1}{(\alpha+x_0)^{\eta-1}} \right).
 \end{aligned}$$

Or, $\int \text{cste} \cdot f^*(x) dx = 1$, avec **cste** la constante de normalisation.

Donc, on a :

$$\text{cste} = \frac{\eta-1}{\left(\frac{2}{\alpha^{\eta-1}} - \frac{1}{(\alpha+x_0)^{\eta-1}} \right)}$$

4.2.1.2 Calcul de F

On a :

$$F(x) = \begin{cases} \int_0^x \frac{\text{cste}}{(\alpha+x_0-t)^\eta} dt & \text{si } 0 \leq x \leq x_0 \\ \int_0^{x_0} \frac{\text{cste}}{(\alpha+x_0-t)^\eta} dt + \int_{x_0}^x \frac{\text{cste}}{(\alpha+t-x_0)^\eta} dt & \text{si } x > x_0 \end{cases}$$

Ainsi, on obtient après des calculs simples :

$$F(x) = \begin{cases} \frac{\text{cste}}{\eta-1} \left[\frac{1}{(\alpha+x_0-x)^{\eta-1}} - \frac{1}{(\alpha+x_0)^{\eta-1}} \right] & \text{si } 0 \leq x \leq x_0 \\ \frac{\text{cste}}{\eta-1} \left[\frac{2}{\alpha^{\eta-1}} - \frac{1}{(\alpha+x_0)^{\eta-1}} - \frac{1}{(\alpha+x-x_0)^{\eta-1}} \right] & \text{si } x > x_0 \end{cases}$$

4.2.2 Détermination de la fonction de répartition inverse

$$* \quad \text{si } x \leq x_0$$

$$F(x) = y$$

$$\Longleftrightarrow \frac{1}{(\alpha + x_0 - x)^{\eta-1}} - \frac{1}{(\alpha + x_0)^{\eta-1}} = \frac{y(\eta-1)}{\text{cste}}$$

$$\Longleftrightarrow x = \alpha + x_0 - \left(\frac{1}{(\alpha + x_0)^{\eta-1}} + \frac{y(\eta-1)}{\text{cste}} \right)^{\frac{-1}{\eta-1}}$$

avec

$$y \in \left[0, \frac{\text{cste}}{\eta-1} \left(\frac{1}{\alpha^{\eta-1}} - \frac{1}{(\alpha + x_0)^{\eta-1}} \right) \right]$$

$$* \quad \text{si } x > x_0$$

$$F(x) = y$$

$$\Longleftrightarrow 2 \cdot \frac{1}{\alpha^{\eta-1}} - \frac{1}{(\alpha + x_0)^{\eta-1}} - \frac{1}{(\alpha + x - x_0)^{\eta-1}} = \frac{y(\eta-1)}{\text{cste}}$$

$$\Longleftrightarrow x = -\alpha + x_0 + \left(2 \cdot \frac{1}{\alpha^{\eta-1}} - \frac{1}{(\alpha + x_0)^{\eta-1}} - \frac{y(\eta-1)}{\text{cste}} \right)^{\frac{-1}{\eta-1}}$$

avec

$$y \in \left[\frac{\text{cste}}{\eta-1} \left(\frac{1}{\alpha^{\eta-1}} - \frac{1}{(\alpha + x_0)^{\eta-1}} \right), 1 \right]$$

Donc on obtient :

$$F^{-1}(y) = \begin{cases} \alpha + x_0 - \left(\frac{1}{(\alpha+x_0)^{\eta-1}} + \frac{y(\eta-1)}{\text{cste}} \right)^{\frac{-1}{\eta-1}} & \text{si } y \in \left[0, \frac{\text{cste}}{\eta-1} \left(\frac{1}{\alpha^{\eta-1}} - \frac{1}{(\alpha+x_0)^{\eta-1}} \right) \right] \\ -\alpha + x_0 + \left(\frac{2}{\alpha^{\eta-1}} - \frac{1}{(\alpha+x_0)^{\eta-1}} - \frac{y(\eta-1)}{\text{cste}} \right)^{\frac{-1}{\eta-1}} & \text{si } y \in \left[\frac{\text{cste}}{\eta-1} \left(\frac{1}{\alpha^{\eta-1}} - \frac{1}{(\alpha+x_0)^{\eta-1}} \right), 1 \right] \end{cases}$$

4.2.3 Création de la fonction rremb()

Pour des raisons d'optimisation de calculs, nous procédons d'abord à une simplification de l'expression de la fonction de répartition inverse.

4.2.3.1 Simplification de F^{-1}

Dans toute la suite, on considérera les variables :

cste_new, **alpha_x0**, **alpha_x0_new**, **alpha_pow**, **eta_minus1**, **inv_eta_minus1**,
définies comme suit :

$$\begin{aligned}\alpha_{x_0} &= \alpha + x_0 \\ \alpha_{x_0_new} &= -\alpha + x_0 \\ \eta_{minus1} &= \eta - 1 \\ \alpha_{pow} &= \frac{1}{\alpha^{\eta-1}} \implies \alpha_{pow} = \alpha^{-\eta_{minus1}} \\ inv_eta_{minus1} &= \frac{1}{\eta_{minus1}} \\ \mathbf{cste_new} &= \frac{\eta - 1}{\mathbf{cste}} \implies \mathbf{cste_new} = 2 \cdot \alpha_{pow} - \alpha_{x_0}^{-\eta_{minus1}}\end{aligned}$$

Ainsi, F^{-1} devient :

$$F^{-1}(y) = \begin{cases} \alpha + x_0 - \left(2 \cdot \frac{1}{\alpha^{\eta-1}} - \frac{(1-y)(\eta-1)}{\mathbf{cste}} \right)^{\frac{-1}{\eta-1}} & \text{si } y \in \left[0, 1 - \frac{\mathbf{cste}}{\eta-1} \cdot \frac{1}{\alpha^{\eta-1}} \right], \\ -\alpha + x_0 + \left(\frac{(1-y)(\mathbf{cste})}{\eta-1} \right)^{\frac{-1}{\eta-1}} & \text{si } y \in \left[1 - \frac{\mathbf{cste}}{\eta-1} \cdot \frac{1}{\alpha^{\eta-1}}, 1 \right]. \end{cases}$$

$$\iff F^{-1}(y) = \begin{cases} \alpha_{x_0} - (2 \cdot \alpha_{pow} - (1-y) \cdot \mathbf{cste_new})^{-inv_eta_minus1} & \text{si } y \in \left[0, 1 - \frac{\alpha_{pow}}{\mathbf{cste_new}} \right], \\ \alpha_{x_0_new} + ((1-y) \cdot \mathbf{cste_new})^{-inv_eta_minus1} & \text{si } y \in \left[1 - \frac{\alpha_{pow}}{\mathbf{cste_new}}, 1 \right]. \end{cases}$$

L'ensemble de définition de F^{-1} étant $[0,1]$, on peut simplifier l'écriture et on obtient :

$$\iff F^{-1}(y) = \begin{cases} \alpha_{x_0} - (2 \cdot \alpha_{pow} - (1-y) \cdot \mathbf{cste_new})^{-inv_eta_minus1} & \text{si } (1-y) \cdot \mathbf{cste_new} \geq \alpha_{pow}, \\ \alpha_{x_0_new} + ((1-y) \cdot \mathbf{cste_new})^{-inv_eta_minus1} & \text{si } (1-y) \cdot \mathbf{cste_new} < \alpha_{pow}. \end{cases}$$

Donc l'écriture finale est :

$F^{-1}(y) = \begin{cases} \alpha_{x_0} - (2 \cdot \alpha_{pow} - (1-y) \cdot \mathbf{cste_new})^{-inv_eta_minus1} & \text{si } (1-y) \cdot \mathbf{cste_new} \geq \alpha_{pow}, \\ \alpha_{x_0_new} + ((1-y) \cdot \mathbf{cste_new})^{-inv_eta_minus1} & \text{si } (1-y) \cdot \mathbf{cste_new} < \alpha_{pow}. \end{cases}$

4.2.3.2 Implémentation de `rremb()`

Pour simuler la loi de remboursement $\mathbb{P}_r$, la méthode suggère de générer un nombre n de valeurs de façon uniforme entre 0 et 1, dont on va calculer l'image par F^{-1} .

Pour cela :

- On stocke dans un vecteur y n valeurs tirées uniformément entre 0 et 1.
- On applique une transformation stratégique avec $y \leftarrow (1 - y) \cdot \text{cste_new}$ en vue d'optimiser la suite du calcul au niveau de F^{-1} , notamment.

En remplaçant $(1-y) \cdot \text{cste_new}$ par y , on obtient une expression plus simple de F^{-1} :

$$F^{-1}(y) = \begin{cases} \alpha_{x0} - (2 \cdot \alpha_{\text{pow}} - y)^{-\text{inv_eta_minus1}} & \text{si } y \geq \alpha_{\text{pow}}, \\ \alpha_{x0_new} + y^{\text{inv_eta_minus1}} & \text{si } y < \alpha_{\text{pow}}. \end{cases}$$

- On utilisera l'une ou l'autre des expressions de F^{-1} en fonction de si la composante de y vérifie ou non la **condition1** :

$$\text{condition1} : y \geq \alpha_{\text{pow}}.$$

- On stocke toutes les valeurs retournées dans un vecteur **Résultats** que retournera la fonction.

```
rremb <- function(n, params) {
  alpha<- params$alpha
  alpha_x0 <- alpha+ params$x0
  alpha_x0_new<- -alpha + params$x0
  eta_minus1 <- params$eta - 1
  alpha_pow <- alpha^(-eta_minus1)

  cste_new <- 2*alpha_pow - (alpha_x0^(-eta_minus1))

  y <- runif(n)
  y<-(1-y)*cste_new
  resultats <- numeric(n)
  inv_eta_minus1 <- 1 / eta_minus1
  condition_1 <- y >=alpha_pow

  resultats[condition_1] <- alpha_x0 - (2*alpha_pow - y[condition_1])^(-inv_eta_minus1)
  resultats[!condition_1] <- alpha_x0_new + y[!condition_1]^(-inv_eta_minus1)
  return(resultats)
}
```

FIGURE 10 – code de *rremb*

- Visualisation de la distribution des valeurs retournées par *rremb* avec les mêmes paramètres qu'en "rejet" :

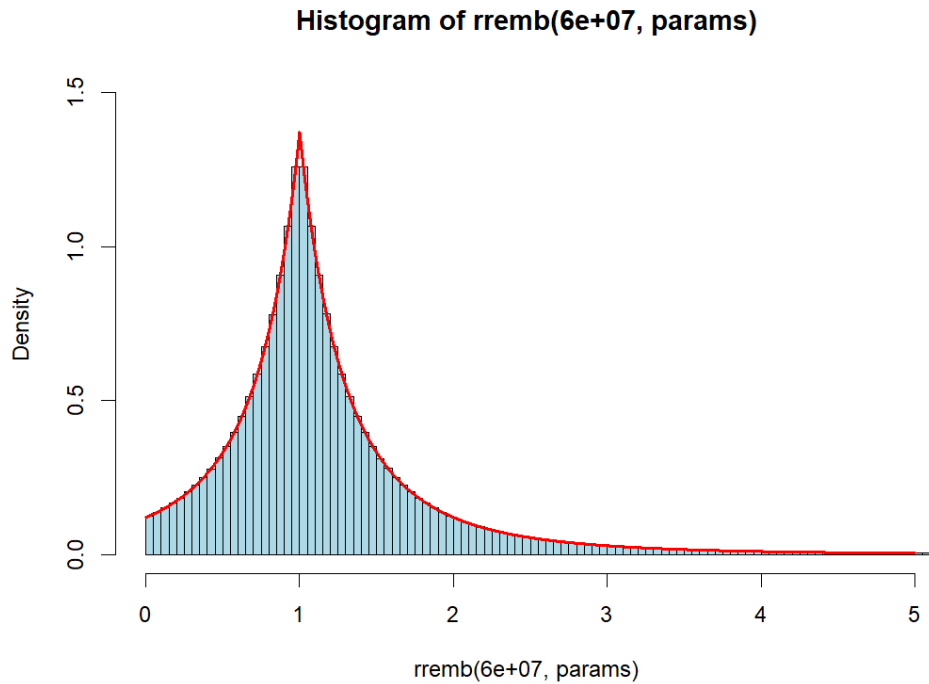


FIGURE 11 – Histogramme de *rremb*

- On remarque d’une part que cette méthode simule mieux la distribution des remboursements que celle par "rejet" et d’autre part, elle est beaucoup plus efficace avec 600 fois plus de simulations en moins de temps :

```
> set.seed(123)
> system.time(rremb(60000000,params))
utilisateur      système      écoulé
          10.67          0.62          11.74
> |
```

FIGURE 12 – Temps d’exécution Méthode par inversion

Ainsi pour la suite de la modélisation, on utilisera le *rremb* obtenu par la méthode d’inversion de la fonction de répartition.

5 Implémentation de *msa* et *msb*

Pour pouvoir simuler *msa* et *msb* nous avons utilisé la méthode de **Monte Carlo**.

5.1 explications préalables

5.1.1 la probabilité d’accidents

Lorsque la météo est connue, chaque jour, on a une probabilité d’avoir un accident qui sera fonction de la météo, et qui sera une loi de Bernoulli pour chacun des motards.

et comme la météo est commune et qu'on a indépendance des accidents entre motards, alors le nombre d'accidents dans la journée suivra une loi binomiale de paramètres, le nombre de motards assurés et la probabilité d'avoir un accident ce jour-là.

5.1.2 l'espérance conditionnelle par monte carlo

Concernant le calcul de l'espérance conditionnelle, on a :

$$\begin{aligned}\mathbb{E}[R - s \mid R > s] &= \mathbb{E}[R \mid R > s] - s \\ \mathbb{E}[R - s \mid R > s] &= \frac{\mathbb{E}[R \times \mathbf{1}_{R>s}]}{\mathbb{P}[R > s]} - s \\ \mathbb{E}[R - s \mid R > s] &= \frac{\frac{\sum_{i=1}^{n.simul} R_i \times \mathbf{1}_{R_i>s}}{n.simul}}{\frac{\sum_{i=1}^{n.simul} \mathbf{1}_{R_i>s}}{n.simul}} - s \\ \mathbb{E}[R - s \mid R > s] &= \frac{\sum_{i=1}^{n.simul} R_i \times \mathbf{1}_{R_i>s}}{\sum_{i=1}^{n.simul} \mathbf{1}_{R_i>s}} - s \\ \mathbb{E}[R - s \mid R > s] &= \frac{\sum_{R_i>s} R_i}{\text{card}\{R_i > s\}} - s \\ \mathbb{E}[R - s \mid R > s] &= \text{mean}(R_i; i \in R_i > s) - s\end{aligned}$$

5.2 La fonction *msa* : explication de l'algorithme

Hypothèse A : les motards circulent tous dans la même région, et ont donc une météo commune.

La fonction **msa** a pour objectif de calculer l'espérance conditionnelle suivante en utilisant une **méthode de Monte Carlo** :

$$m(s) = \mathbb{E}(R - s \mid R > s)$$

où :

- R représente le montant total des remboursements ;
- s est un seuil donné.

5.2.1 Initialisation des variables :

La fonction extrait les paramètres suivants de la liste **params** :

- N : Le nombre de motards assurés ;
- s : Le seuil de remboursement ;
- $\text{days} = 365$: La période d'étude ;
- **pacc** : Les probabilités d'accident pour chaque météo (p_S, p_N, p_P) ;
- **ptrans** : Les probabilités de transition de la chaîne de Markov.

Pour les besoins du calcul, on initialise 2 vecteurs :

- **R_values** : Contenant les remboursements ($R - s$) pour chaque simulation ;
- **prob_accident** : Contenant les probabilités journalières d'accident en fonction de la météo.

5.2.2 Boucle de simulation Monte Carlo

Pour réaliser des simulations indépendantes, la boucle principale est exécutée `n.simul` fois, où `n.simul` représente le nombre de simulations.

Étape 1 : Simulation de la météo

La fonction `simulate_weather_vectorized` génère un vecteur `weather` de taille 365, représentant la météo journalière sur une année :

- 1 : Soleil ;
- 2 : Nuages ;
- 3 : Pluie.

Étape 2 : Assignment des probabilités d'accident

Pour chaque jour, On assigne les probabilités d'accidents selon la météo :

$$\text{prob_accident}[t] = \begin{cases} p_{\text{acc}}[1] & \text{si météo} = \text{Soleil} \\ p_{\text{acc}}[2] & \text{si météo} = \text{Nuages} \\ p_{\text{acc}}[3] & \text{si météo} = \text{Pluie} \end{cases}$$

Étape 3 : Simulation du nombre d'accidents

Le nombre d'accidents journalier est simulé à l'aide d'une loi binomiale :

$$\text{nombre d'accidents par jour} \sim \mathcal{B}(N, \text{prob_accident}[t])$$

Le résultat est un vecteur de taille 365 contenant le nombre d'accidents par jour.

Étape 4 : Simulation des remboursements

Nous calculons la somme des accidents pour obtenir le nombre total d'accidents annuels :

$$\text{total accidents} = \sum_{t=1}^{365} \text{nombre d'accidents}[t]$$

Nous utilisons la fonction `rremb` pour simuler les montants de remboursement associés à ces accidents.

Nous calculons le remboursement total que nous ajustons avec le seuil s :

$$R_{\text{net}} = R - s$$

Cette valeur est stockée dans le vecteur `R_values`.

5.2.3 Calcul de l'espérance conditionnelle

Après avoir exécuté toutes les simulations :

- On filtre les valeurs de `R_values` pour ne conserver que celles strictement positives ($R > s$).
- L'espérance conditionnelle est alors calculée comme la moyenne des valeurs restantes :

$$m(s) = \frac{1}{n} \sum_{i=1}^n R_{\text{net},i}, \quad \text{avec } R_{\text{net},i} > 0$$

- L'écart-type des valeurs filtrées est utilisé pour calculer la demi-largeur de l'intervalle de confiance à 95% :

$$\text{Demi-largeur} = 1.96 \times \frac{\sigma}{\sqrt{n}}$$

5.2.4 valeur de sortie de la fonction msa

La fonction retourne une liste contenant :

- `ms` : L'espérance conditionnelle $m(s)$;
- `demi.largeur` : La demi-largeur de l'intervalle de confiance à 95%.

```
msa <- function(n.simul, params) {
  #nombre de motards
  N <- params$N
  #seuil
  s <- params$s
  #nombre de jours
  days <- 365
  #probabilités de transition
  ptrans <- params$ptrans
  #probabilités d'accidents
  pacc <- params$pacc

  R_values <- numeric(n.simul)

  prob_accident <- numeric(days)

  for (sim in 1:n.simul) {
    #creation d'un vecteur de taille days avec les meteos pour chaque jour
    weather <- simulate_weather_vectorized(days, params)

    #on affecte en fonction de la meteo les probabilités d'accident
    prob_accident[weather==1] <- pacc[1]
    prob_accident[weather==2] <- pacc[2]
    prob_accident[weather==3] <- pacc[3]

    #on recupere le nombre d'accident qui on eu lieu pour les days jours
    num_accidents_per_day <- rbinom(days, N, prob_accident)

    #on somme pour obtenir le nombre d'accident pour l'annee
    tot <- sum(num_accidents_per_day)

    #on applique la fonction cremb pour avoir le remboursement pour chacun des accidents
    remboursements <- rremb(tot, params)

    R_values[sim] <- sum(remboursements)
  }

  # Calcul de l'espérance conditionnelle  $m(s) = E(R - s | R > s)$ 
  ms1 <- R_values[R_values > s]
  #moyenne
  m_s <- mean(ms1) - s
  #ecart type
  sigma <- sd(ms1)

  n <- length(ms1)

  return(list(ms = m_s, demi.largeur=(1.96*sigma/sqrt(n))))
}
```

FIGURE 13 – code de *msa*

l'exécution du code donne prend en compte les paramètres suivant :

```
params <- list(
  x0=1,
  alpha=1,
  eta=3.5,
  N = 5000,
  s = 1000,
  ptrans = c(0.1, 0.1, 0.1,0.1), # Probabilités de transition (exemple)
  pacc = c(0.01, 0.02, 0.03) # Probabilités d'accidents (exemple)
)

set.seed(123)
system.time(a<-msa(4000,params))
a
```

FIGURE 14 – Paramètres d'exécution de l'exemple

Et on obtient :

```

> set.seed(123)
> system.time(a<-msa(4000,params))
utilisateur      système      écoulé
      27.18         0.36        28.04
> a
$ms
[1] 43174.54

$demi.largeur
[1] 126.5562

```

FIGURE 15 – Résultats de la fonction et Temps d'exécution de `msa`

5.3 La fonction `msb` : explication de l'algorithme

5.3.1 Initialisation des variables

La fonction extrait les paramètres suivants de la liste `params` :

- N : Le nombre de motards assurés ;
 - s : Le seuil de remboursement ;
 - $\text{days} = 365$: La période d'étude ;
 - pacc : Les probabilités d'accident pour chaque météo (p_S, p_N, p_P) ;
 - ptrans : Les probabilités de transition de la chaîne de Markov.
- Les états (S,N,P) étant qualitatifs, le travail devra se faire avec des variables alternatives qui reposeront sur l'écart entre deux variables successives du même état, ces variables seront donc générées avec la logique suivante :
- Calcul des indices pour chaque état :
 - Pour l'état S :
 - `indices_1` : indices des valeurs 1 dans `weather`, calculé avec `which(weather == 1)`.
 - `diff_indices_1` : différences successives des indices d'apparition de 1, calculé avec `diff(indices_1)`.
 - Pour l'état N :
 - `indices_2` : indices des valeurs 2 dans `weather`, calculé avec `which(weather == 2)`.
 - `diff_indices_2` : différences successives des indices d'apparition de 2, calculé avec `diff(indices_2)`.
 - Pour l'état P :
 - `indices_3` : indices des valeurs 3 dans `weather`, calculé avec `which(weather == 3)`.
 - `diff_indices_3` : différences successives des indices d'apparition de 3, calculé avec `diff(indices_3)`.
 - Création des variances estimées pour chaque état :
 - Variance estimée pour l'état S : $Vn_S = \text{sd}(\text{diff_indices_1})$.
 - Variance estimée pour l'état N : $Vn_N = \text{sd}(\text{diff_indices_2})$.
 - Variance estimée pour l'état P : $Vn_P = \text{sd}(\text{diff_indices_3})$.
- Pour les besoins du calcul, on initialise :
- `R_values` : Contenant les remboursements ($R - s$) pour chaque simulation.

5.3.2 Problématique par rapport à la méthode initiale

La méthode initiale était assez problématique dans la mesure où appliquée à la fonction `msb` la vitesse était trop faible due au fait que l'appel de la fonction `simulate_weather_vectorized` `n.simul` fois était trop coûteux en terme de temps et donc ne permettait pas de satisfaire les exigences du site de soumission.

```
msb <- function(n.simul, params) {
  N <- params$N
  s <- params$s
  days <- 365
  ptrans <- params$ptrans
  pacc <- params$pacc

  R_values <- numeric(n.simul)
  a <- 0
  for (sin in 1:n.simul) {
    a <- append(a, system.time(weather <- matrix(unlist(lapply(1:N, function(x) simulate_weather_vectorized(days, params))), nrow = N, byrow = TRUE)))

    # Calculer les probabilités d'accidents pour chaque motard et chaque jour
    prob_accident <- matrix(0, nrow = N, ncol = days)
    prob_accident[weather == 1] <- pacc[1]
    prob_accident[weather == 2] <- pacc[2]
    prob_accident[weather == 3] <- pacc[3]

    # Simuler les accidents pour tous les motards en une seule opération
    accidents <- rbinom(N * days, 1, prob_accident)

    num_accidents <- sum(accidents)

    R_values[sin] <- sum(rrexp(num_accidents, params)) - s
  }

  # Calcul de l'espérance conditionnelle m(s) = E(R - s | R > s)
  ms1 <- R_values[R_values > 0]
  m_s <- mean(ms1)

  sigma <- sd(ms1)
  n <- length(ms1)

  return(list(ms = m_s, demi.largeur = (1.96*sigma/sqrt(n)), a))
}
```

FIGURE 16 – script initial de la fonction `msb`

C'est en cela que nous avons élaboré une autre méthode afin de pouvoir palier ce problème.

5.3.3 Méthode alternative

Pour réaliser des simulations indépendantes, la boucle principale est exécutée `n.simul` fois, où `n.simul` représente le nombre de simulations.

Étape 1 : Simulation de la météo par régions

```
weather = matrix(simulate_weather_vectorized(days, params), nrow = N, byrow = TRUE)
```

On appelle une fois la fonction `simulate_weather_vectorized` génère une matrice `weather` représentant la météo pour `days` jours pour un seul motard, afin d'avoir un état initial sur lequel baser le reste des simulations (ce qui est faisable car pour chaque motard on a les mêmes probabilités de transition dans la chaîne de Markov)

Étape 2 : Calcul des statistiques météorologiques

$$nbS = \text{sum}(\text{weather} == 1)$$

$$nbN = \text{sum}(\text{weather} == 2)$$

$$nbP = \text{sum}(\text{weather} == 3)$$

On calcule les statistiques pour chaque type de météo (Soleil, Nuages, Pluie).

Étape 3 : Simulation des perturbations aléatoires

Pour chaque simulation, nous introduisons des perturbations aléatoires :

$$\begin{aligned}\epsilon_S &= \text{runif}(1, -Vn_S, Vn_S) \\ \epsilon_N &= \text{runif}(1, -Vn_N, Vn_N) \\ \epsilon_P &= -(\epsilon_S + \epsilon_N)\end{aligned}$$

Remarque : pour conserver le nombre d'élément final qui correspond au nombre de jour nous avons défini ϵ_P de la sorte.

Étape 4 : Création des niveaux météorologiques avec perturbations

```
weather = factor(sample(rep(labels, times = repetitions)))
```

Où labels = (1, 2, 3) et repetitions = (nbS + ϵ_S , nbN + ϵ_N , nbP + ϵ_P).

La logique derrière la conception de **repetition** est que chaque motard n'aura pas forcément le même nombre de S de N ou de P, mais étant donné que les probabilités de transition sont les mêmes, ces quantités ne varieront pas énormément entre deux motards. D'où l'ajout de perturbations aléatoires.

Étape 5 : Simulation des accidents et calcul des remboursements

```
prob_accident = matrix(0, nrow = N, ncol = days)
prob_accident[weather == 1] = p_acc[1]
prob_accident[weather == 2] = p_acc[2]
prob_accident[weather == 3] = p_acc[3]

accidents = rbinom(N * days, 1, prob_accident)
```

Le nombre total d'accidents annuels est :

```
num_accidents = sum(accidents)
```

Les remboursements associés sont calculés à l'aide de la fonction **rremb**.

Étape 6 : Calcul du remboursement net

Nous calculons le remboursement total ajusté avec le seuil s :

$$R_{\text{net}} = \sum (\text{rremb}(\text{num_accidents}, \text{params})) - s$$

Cette valeur est stockée dans le vecteur **R_values**.

5.3.4 Calcul de l'espérance conditionnelle

Après avoir exécuté toutes les simulations :

- On filtre les valeurs de **R_values** pour ne conserver que celles strictement positives ($R > s$).
- L'espérance conditionnelle est alors calculée comme la moyenne des valeurs restantes :

$$m(s) = \frac{1}{n} \sum_{i=1}^n R_{\text{net},i}, \quad \text{avec } R_{\text{net},i} > 0$$

- L'écart-type des valeurs filtrées est utilisé pour calculer la demi-largeur de l'intervalle de confiance à 95% :

$$\text{Demi-largeur} = 1.96 \times \frac{\sigma}{\sqrt{n}}$$

5.3.5 Valeur de sortie de la fonction *msb*

La fonction retourne une liste contenant :

- `ms` : L'espérance conditionnelle $m(s)$;
- `demi.largeur` : La demi-largeur de l'intervalle de confiance à 95%.

6 Interfaçage de *msb* en C++

6.1 Algorithmie

La logique derrière le code reste inchangée, mais le segment suivant a été retranscrit en C++, car constituant la partie la plus chronophage :

```
weather <-simulate_weather_vectorized(days, params)
nbS<-sum(weather==1)
nbN<-sum(weather==2)
nbP<-sum(weather==3)
# Indices des 1
indices_1 <- which(weather == 1)
diff_indices_1 <- diff(indices_1)
# Indices des 2
indices_2 <- which(weather == 2)
diff_indices_2 <- diff(indices_2)
# Indices des 3
indices_3 <- which(weather == 3)
diff_indices_3 <- diff(indices_3)
```

FIGURE 17 – Partie n°1 *msb.c*

Ce qui a engendré le code suivant :

```
msb.ct<-function(n.simul,params){
  N <- params$N
  s <- params$s
  days <- 365
  ptrans <- params$ptrans
  pacc <- params$pacc
  R_values <- numeric(n.simul)
  cppFunction("List msbc(int days,List params,NumericVector ptrans,Function simulate_weather_vectorized){
    RNGScope scope;
    NumericVector weather=simulate_weather_vectorized(days,params);
    int nbS = std::count(weather.begin(), weather.end(), 1);
    int nbN = std::count(weather.begin(), weather.end(), 2);
    int nbP = std::count(weather.begin(), weather.end(), 3);
    //code pour remplacer le which de R :

    std::vector<int> indices_1=which_indices(weather,1);
    std::vector<int> indices_2=which_indices(weather,2);
    std::vector<int> indices_3=which_indices(weather,3);
    //code pour remplacer le diff de R :

    std::vector<int> diff_indices_1=diff_indices(indices_1);
    std::vector<int> diff_indices_2=diff_indices(indices_2);
    std::vector<int> diff_indices_3=diff_indices(indices_3);

    return List::create(
      _["diff_indices_1"] = diff_indices_1,
      _["diff_indices_2"] = diff_indices_2,
      _["diff_indices_3"] = diff_indices_3,
      _["nbr de S"]=nbS,
      _["nbr de N"]=nbN,
      _["nbr de P"]=nbP
    );
  }");
}
```

FIGURE 18 – Partie n°2 *msb.c*

Dans la portion précédente, on retourne une liste composée de six éléments dont on renomme les emplacements avec des noms qui définissent chaque élément pour l'appel de ces derniers dans la suite du code :

```
.includes<std::vector<int>> which_indices(const NumericVector& weather, int etat) {
    std::vector<int> indices;
    for (int i = 0; i < weather.size(); ++i) {
        if (weather[i] == etat) {
            indices.push_back(i);
        }
    }
    return indices;
}

std::vector<int> diff_indices(const std::vector<int>& indices) {
    std::vector<int> diffs(indices.size() - 1);
    for (size_t i = 1; i < indices.size(); ++i) {
        diffs[i - 1] = indices[i] - indices[i - 1];
    }
    return diffs;
}

results<-msbc(days, params, ptrans, simulate_weather_vectorized)

Vn_S<-sd(results[["diff_indices_1"]])
Vn_M<-sd(results[["diff_indices_2"]])
Vn_P<-sd(results[["diff_indices_3"]])
nbS<-results[["nbr de S"]]
nbM<-results[["nbr de M"]]
nbP<-results[["nbr de P"]]
for (sim in 1:n.simul) {
    epsilon_S<-runif(1, -Vn_S, Vn_S)
    epsilon_M<-runif(1, -Vn_M, Vn_M)
    epsilon_P<- -epsilon_S+epsilon_M #! faut conserver une relation lineaire entre les epsilons pour etre sur qu'd la fin on ait toujours 365 éléments po
    labels <- c(1,2,2)
    repetitions <- c(nbS+epsilon_S, nbM+epsilon_M, nbP+epsilon_P)
    # Creation des niveaux avec répétitions spécifiques
    weather <- factor(rep(labels, times = repetitions))
}
```

FIGURE 19 – Partie n°3 msb.c

Dans le "includes", on a défini les fonctions intermédiaires qui ont été utilisées à l'intérieur de "cppFunction".

À partir des noms d'emplacement définis plus haut, nous avons assigné aux différentes variables avec lesquelles nous allons travailler par la suite, les valeurs retournées par la fonction C++.

La partie suivante n'est que l'ajout du code préexistant dans msb , c'est-à-dire sans modifications.

```
# Affichage
# Calculer les probabilités d'accidents pour chaque motard et chaque jour
prob_accident <- matrix(0, nrow = N, ncol = days)
prob_accident[weather == 1] <- pacc[1]
prob_accident[weather == 2] <- pacc[2]
prob_accident[weather == 3] <- pacc[3]

# Simuler les accidents pour tous les motards en une seule opération
accidents <- rbinom(N * days, 1, prob_accident)

num_accidents <- sum(accidents)

R_values[sim] <- sum(rremb(num_accidents, params)) - s
}

# Calcul de l'espérance conditionnelle m(s) = E(R - s | R > s)
ms1 <- R_values[R_values > 0]
m_s <- mean(ms1)

sigma<-sd(ms1)
n<-length(ms1)

return(list(ms = m_s, demi.largeur=(1.96*sigma/sqrt(n))))
```

FIGURE 20 – Partie n°4 msb.c

Remarque : Le code ci-dessous représente une tentative d'implémentation de la boucle "for" du "msb" en C++, mais l'idée a été abandonnée, car les résultats avaient une variance et une demi-largeur plus grande que celles du code précédent (celui sans retranscription de la boucle en C++), pour un gain en temps négligeable.

```

cppFunction("R_matrix_smbc(int n_simu,List params, double vn_5,double vn_N,int N,int s,int days,MatrixVector pacc,int nb5,int nb6,int nbP, Function rverb){
  Rcpp::Export();
  MatrixVector R_values(n_simu);
  for (int i=0;i<n_simu;i++){
    double epsilon_5 = R::runif(0,vn_5, vn_5);
    double epsilon_N = R::runif(0,vn_N, vn_N);
    double epsilon_P = R::expm1(-epsilon_5 + epsilon_N);
    MatrixVector repetitions(1);
    repetitions[0] = nb5 + epsilon_N*N;
    repetitions[0] = nb6 + epsilon_N*N;
    repetitions[0] = nbP + epsilon_P;
    std::vector<int> weather;
    for (int j = 0; j < repetitions[0]; j++) {
      weather.push_back(1 + j); // Rsample avec 1, 2, ou 3 pour 5, N, P
    }
    MatrixMatrix prob_accident(N, days);
    for (int i = 0; i < N; i++) {
      for (int j = 0; j < days; j++) {
        if (weather[j] == 1) {
          prob_accident(i, j) = pacc[0];
        } else if (weather[j] == 2) {
          prob_accident(i, j) = pacc[1];
        } else {
          prob_accident(i, j) = pacc[2];
        }
      }
    }
    MatrixMatrix accident(N, days);
    for (int i = 0; i < N; i++) {
      for (int j = 0; j < days; j++) {
        accident(i, j) = R::rbinom(1, prob_accident(i, j));
      }
    }
    int num_accidents;
    for(int i=0;i<repetitions[0];i++){
      num_accidents = accident[i]; // Ajouter chaque valeur à num_accidents
    }
    MatrixVector rverb_values_P = rverb(num_accidents, params);
    std::vector<double> rverb_values = Rcpp::as<std::vector<double>>(rverb_values_P);
    int sum_rverb;
    for (int i=0;i<repetitions[0];i++){
      sum_rverb=sum_rverb+values[i];
    }
    R_values[i]=sum_rverb/s;
  }
  return R_values;
}
R_values=smbc(n_simu,params,vn_5,vn_N,N,s, days,pacc,nb5,nb6,nbP,rverb)

```

FIGURE 21 – Partie n°5 msb.c

6.2 Résultats

Compte tenu des explications ci-dessus, on obtient les résultats suivants :

```
> system.time(msb(13, params))
```

utilisateur	système	écoulé
1.16	0.03	2.56

FIGURE 22 – Temps d'exécution n°1 msb

```
> system.time(msb.c(13, params))
```

utilisateur	système	écoulé
1.14	0.05	2.36

FIGURE 23 – Temps d'exécution n°1 msb.c

On constate que l'écart entre les deux versions n'est pas significatif pour 13 simulations.

Cependant, quand on passe à 1000 simulations, la différence entre les deux temps se révèle significative. Ainsi, on en déduit que la méthode **cppFunction** du package **Rcpp** est nettement plus efficace que la version de base sur R et que la différence de temps entre les deux versions n'est pas **linéaire**.

```
> system.time(msb(1000,params))
```

utilisateur	système	écoulé
112.55	3.54	375.12

FIGURE 24 – Temps d'exécution n°2 msb

```
> system.time(msb.c(1000, params))
```

utilisateur	système	écoulé
102.39	3.63	296.59

FIGURE 25 – Temps d'exécution n°2 msb.c

En ce qui concerne les valeurs renvoyées, elles ne sont pas exactement les mêmes mais sont relativement proches à seed fixée.

On obtient pour **msb** :

```
$ms  
[1] 122945  
  
$demi.largeur  
[1] 135.015
```

FIGURE 26 – Valeurs $m(s)$ avec msb

Et pour **msb.c**, on a :

```
$ms  
[1] 122784  
  
$demi.largeur  
[1] 137.6298
```

FIGURE 27 – Valeurs $m(s)$ avec msb.c

7 Analyse du modèle

L'objectif de cette section est d'étudier l'influence de certains paramètres en particulier sur le modèle et de mettre en avant quelques études basées sur ce modèle qui pourraient être pertinentes.

7.1 Influence des paramètres

Dans cette sous section nous avons étudié l'influence de la variation des différents paramètres que sont :

- p_P la probabilité d'avoir un accident quand il pleut
- x_0
- η
- s
- p_{NP}

en fixant les autres paramètres aux valeurs qui ont été choisies dans la section 2 de ce document.

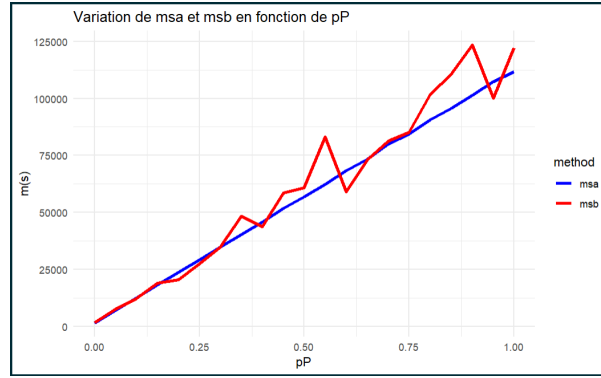


FIGURE 28 – variation de msa et msb en fonction de la probabilité de d'accident en cas de pluie

On constate que lorsque la probabilité de faire un accident lorsqu'il pleut augmente, la valeur de msa et msb augmente aussi, on observe qu'il y'a une relation linéaire croissante entre la probabilité de faire un accident lorsqu'il pleut et la valeur de $m(s)$, ce qui est assez contre intuitif pour ce qui concerne la linéarité.

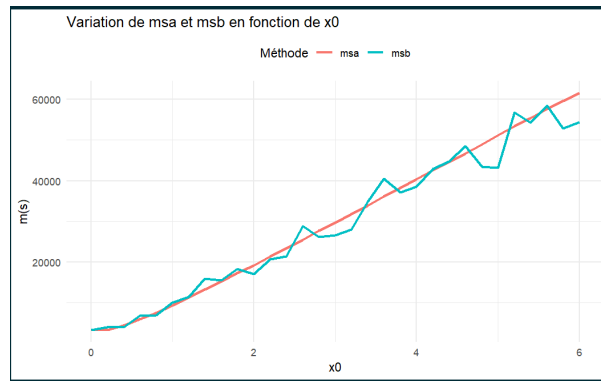


FIGURE 29 – Variation de msa et msb en fonction du paramètre x_0

Pour ce qui est de la variation de x_0 , on a également une relation croissante entre x_0 et msa et msb , ce qui n'était pas vraiment prévisible car la distribution de $rremb$ est centrée en x_0 et se décale juste quand ce paramètre évolue.

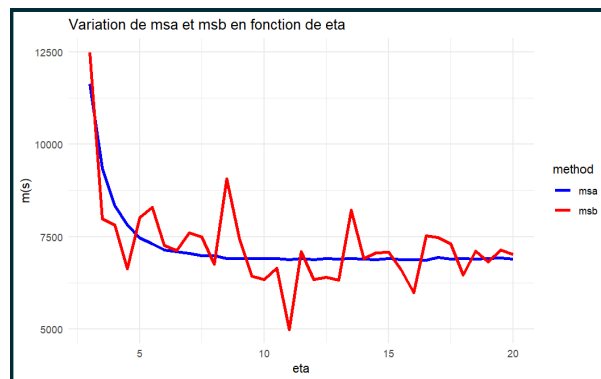


FIGURE 30 – variation de msa et msb en fonction du paramètre η

Concernant la variation de msa et msb , plus η augmente, plus msa et msb diminuent, cette interprétation était plus ou moins attendue car l'augmentation de la valeur de η a tendance à aplatiser la distribution de $rremb$, (dû au fait que la fonction est de la forme $(\frac{1}{x})^\eta$).

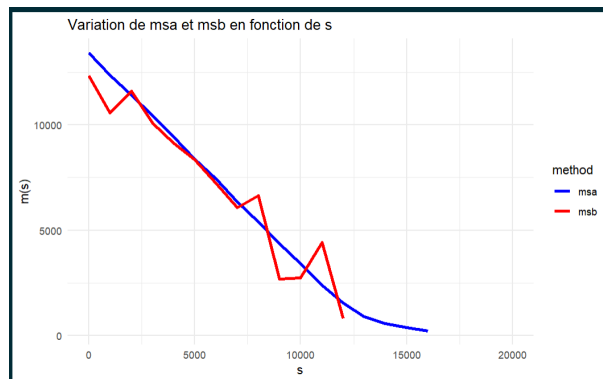


FIGURE 31 – variation de msa et msb en fonction du seuil s

La variation négative de msa et msb en fonction du seuil était quant-à-elle attendue, car c'est plutôt logique que le montant de remboursement supérieur au seuil diminue lorsque la valeur du seuil augmente.

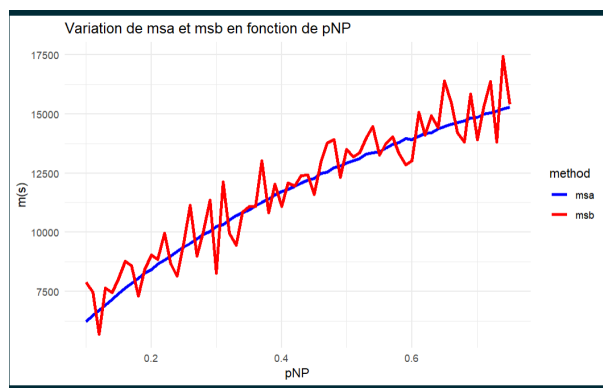


FIGURE 32 – variation de msa et msb en fonction de p_{NP}

Et pour finir, l'augmentation de la probabilité de transition de l'état nuageux à l'état pluvieux entraîne une augmentation de la valeur de $m(s)$ ce qui est prévisible car on passe beaucoup plus à l'état puvieux dans lequel la probabilité de faire un accident est élevée, et plus la valeur de la probabilité de faire un accident lorsqu'il pleut augmente, plus la vitesse de croissance est élevée.

7.2 Cas d'application du modèle

Ce modèle pourrait nous servir dans plusieurs cas d'études, cependant, nous avons cependant décidé de n'en retenir que deux :

- L'Assureur peut s'en servir comme une aide à la décision pour savoir si nous devons diversifier le portefeuille d'assuré, en choisissant de rester à un niveau plus local ou assurer plusieurs zones distinctes.

Par exemple : Considérons un assureur proposant un produit d'assurance moto pour des motards, un domaine en forte croissance et où l'assureur prévoit attirer 1000 assurés de plus.

Pour des besoins stratégiques il souhaite savoir s'il devrait intensifier la publicité au niveau local ou l'étendre à d'autres régions sachant que le budget sera le même dans tous les cas.

L'historique météorologique national montre que les probabilités de transitions météorologiques et d'accidents sont celle fixées dans la partie **2.3** et le seuil fixé à 1000 par l'assureur.

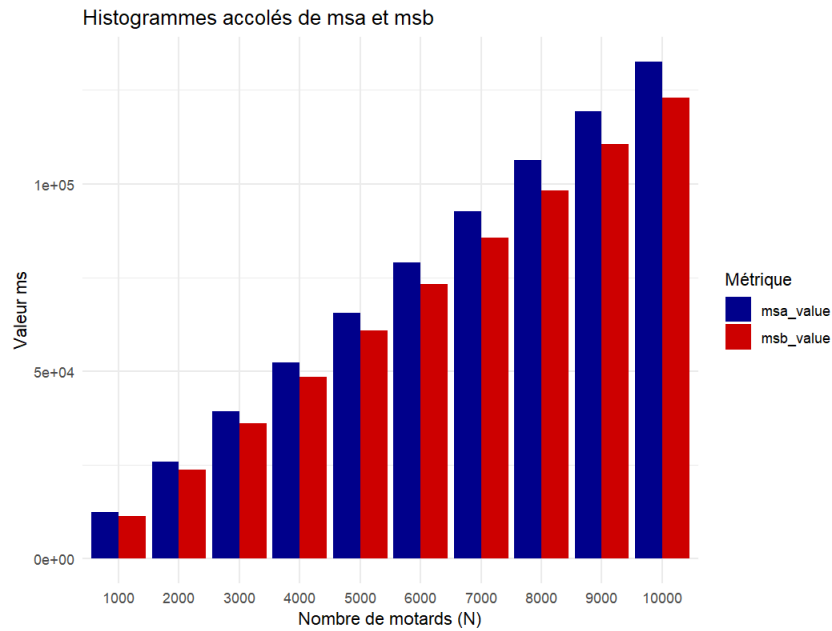


FIGURE 33 – Histogramme comparatif des différents remboursements pour chaque hypothèse.

Interprétation : Si l'assureur possède un portefeuille de 5000 assurés et considère comme sinistre majeur, les sinistres dont le coût excède 1000 euros, le graphique précédent suggère qu'il serait économiquement plus judicieux de diversifier le portefeuille en étendant ce produit d'assurance à d'autres régions.

En effet ouvrir cette couverture d'assurance à d'autre régions coûterait 5654 euros de moins que dans l'hypothèse où il demeure dans une zone unique lorsque la taille du portefeuille passe de 5000 à 6000 assurés.

	N	msa_value	msb_value
1	1000	12406.28	11386.18
2	2000	25755.99	23784.33
3	3000	39200.93	36175.00
4	4000	52220.58	48567.93
5	5000	65632.30	60902.87
6	6000	78964.75	73318.40
7	7000	92630.04	85666.27
8	8000	106288.32	98093.07
9	9000	119345.38	110495.63
10	10000	132454.16	122945.02

FIGURE 34 – Table comparative des différents remboursements pour chaque hypothèse

- Cette étude peut également permettre à l'assureur de négocier un niveau de **franchise** intéressant (ce qui correspond au seuil s dans notre modèle d'étude) pendant la période de renouvellement de réassurance.
- En effet, en réassurance non proportionnelle, le réassureur n'intervient que lorsque le montant de sinistre excède un montant minimum appelé la franchise, qui est définie par exemple dans un traité excédant de sinistre.

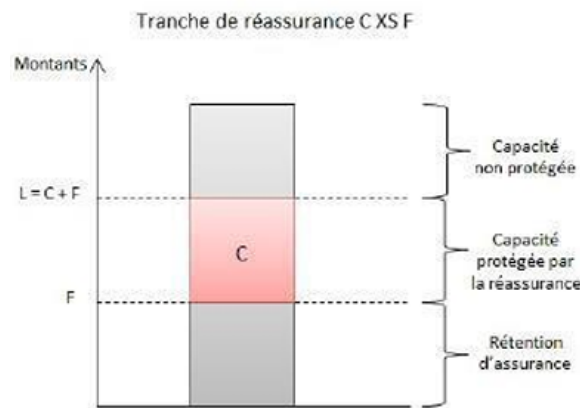


FIGURE 35 – Fonctionnement Excédent de sinistre

7.3 Pistes d'amélioration

- Le modèle devrait inclure la possibilité qu'un assuré puisse avoir plus d'un accident par jour.
- Dans la pratique la météo n'est pas une chaîne de markov avec impossibilité de passer de soleil à pluie ou inversement.
- On pourrait tenir compte des saisons dans la mesure où le verglas sur les routes pourrait augmenter la probabilité d'avoir un accident en fonction de la météo.
- Il serait intéressant d'analyser les probabilités d'accident en fonction du moment de la journée, de la zone géographique, de l'historique d'accident de l'assuré, de la démographie, de son âge, du respect des mesures de sécurité et du type d'engin

- qu'il conduit.
- Prendre comme météo initiale celle du jour où sera faite la simulation en vue de la prédiction.